Arquitectura de SafeGuard

Clientes

Sitio web (Vercel)
React 19 + Vite, https://safeguardd.vercel.app
PhishGuard: empleados, campañas, tablero, informe y cobro
SafeLink: enlace, WhatsApp, correo, PDF, QR, historial, Faro y compartir
Detector propio: analiza la dirección en el navegador; el PDF y el QR no se suben

App Android (Java, WebView)
Carga el sitio publicado
Cámara para QR, compartir texto, enlace copiado y aviso sin conexión

Página de simulación
/simulacion/:token y /bien/:token
la abre el empleado, sin cuenta

Extensión de Chrome (MV3)
Copia del detector: color por pestaña
Avisa, no bloquea, sin historial

GitHub Actions
npm test, eval del detector y build en cada cambio; Vercel publica el sitio

Supabase

Auth
Correo y contraseña o Google
La contraseña se guarda como hash
Emite el JWT de la sesión

Edge Functions (TypeScript, Deno)
safelink-enriquecer: sigue el enlace
safelink-faro: explica el resultado
phishguard-avisar-campana: correo y kit
phishguard-avisar-caida: aviso al admin
phishguard-mercadopago: cobro
eliminar-cuenta: borra la cuenta
Validan la sesión con auth.getUser()

PostgreSQL con seguridad por fila
PhishGuard: organizaciones, usuarios_admin, empleados, plantillas_phishing, campanas, eventos_simulacion, pagos_phishguard
SafeLink: safelink_analisis, safelink_reportes, safelink_compartidos, safelink_errores, safelink_faro_uso, entrenamiento_respuestas
Compartida: amenazas
Funciones: phishguard_registrar, phishguard_tablero, importar empleados

Servicios externos

Google
Ingreso con Google (OAuth)

Gmail (SMTP)
Correo de la campaña, kit y aviso de caída

Mercado Pago
Cobro del mínimo y consulta del pago

Google Gemini
Modelo de Faro: solo el caso y la pregunta

rdap.org, crt.sh y DNS de Cloudflare
Edad del dominio y del certificado

Sitio que se revisa
Se abre desde el servidor, sin cookies

sesión
invoca
carga el sitio
datos con RLS
clave de servicio
registra el evento
lee amenazas
revisa el enlace